Java Errors

Even experienced Java developers make mistakes. The key is learning how to **spot** and **fix** them!

These pages cover common errors and helpful debugging tips to help you understand what's going wrong and how to fix it.

Types of Errors in Java

Error Type	Description
Compile-Time Error	Detected by the compiler. Prevents code from running.
Runtime Error	Occurs while the program is running. Often causes crashes.
Logical Error	Code runs but gives incorrect results. Hardest to find.

Common Compile-Time Errors

Compile-time errors occur when the program cannot compile due to syntax or type issues.

Here are some examples:

1) Missing Semicolon

Example

```
int x = 5

System.out.println(x);
```

Result:

```
error: ';' expected
```

2) Undeclared Variables

Example

```
System.out.println(myVar);
```

Result:

```
cannot find symbol
  symbol: variable myVar
```

3) Mismatched Types

Example

```
int x = "Hello";
```

Result:

```
incompatible types: String cannot be converted to int
```

Common Runtime Errors

Runtime errors occur when the program compiles but crashes or behaves unexpectedly.

Here are some examples:

1) Division by Zero

Example

```
int x = 10;

int y = 0;

int result = x / y;

System.out.println(result);
```

Result:

```
Exception in thread "main" java.lang.A
```

2) Array Index Out of Bounds

Example

```
int[] numbers = {1, 2, 3};

System.out.println(numbers[8]);
```

Result:

```
Exception in thread "main"
java.lang.ArrayIndexOutOfBoundsException: Index 8 out of
bounds for length 3
```

Logical Errors

Logical errors happen when the code runs, but the result is not what you thought:

Example:

```
int x = 10;

int y = 2;

int sum = x - y;
```

```
System.out.println("x + y = " + sum);
```

Result:

```
x + y = 8
```

Good Habits to Avoid Errors

- Use meaningful variable names
- Read the error message carefully. What line does it mention?
- Check for missing semicolons or braces
- Look for typos in variable or method names

In the next chapter, you will learn how to **debug your code** - how to find and fix bugs/errors in your program.

ERROR IN WHATSAPP LANGUAGE----

● WHAT IS ERROR

Error ka matlab hota hai
☞ program mein aisi **serious problem** aa jana
☞ jisse program **properly run hi nahi kar pata**

Simple words mein:

Error = badi problem jo program khud handle nahi kar sakta

◆ Error ko handle kyun nahi kar sakte?

- Error JVM level ki problem hoti hai
- Program ke control ke bahar hoti hai
- Isliye try–catch se **usually handle nahi hoti**

◆ Types of Errors

1️⃣ Compile Time Error

☞ Coding likhte time aati hai
☞ Jab syntax galat hota hai

Examples:

- semicolon `;` missing
- spelling mistake
- wrong datatype

📌 *Compiler error dikha deta hai, program run hi nahi hota*

2️⃣ Runtime Error

☞ Program run karte waqt aati hai
☞ Galat logic ki wajah se

Examples:

- divide by zero
- array ka wrong index
- null object use karna

📌 *Isko exception handling se control kar sakte hain*

3️⃣ Logical Error

☞ Program run hota hai
☞ Par output galat aata hai

Examples:

- formula galat
- condition galat likhna

📌 *Ye sabse dangerous hota hai kyunki error show nahi hoti*

4️⃣ System Error

☞ System / JVM / Hardware se related hoti hai
☞ Program ke bas ki baat nahi hoti

Examples:

- OutOfMemoryError
- StackOverflowError

📌 *Isko fix karne ke liye system resource badhane padte hain*

Java Debugging

After learning about common errors, the next step is understanding how to **debug** your Java code - that is, how to find and fix those errors effectively.

This page introduces simple debugging techniques that are useful for beginners and helpful even for experienced developers.

What is Debugging?

Debugging is the process of identifying and fixing errors or bugs in your code.

It often involves:

- Reading error messages
- Tracing variable values step by step
- Testing small pieces of code independently

Tip: Debugging is a skill that improves with practice. The more you debug, the better you get at spotting problems quickly.

Print Statements for Debugging

The most basic (and often most effective) way to debug Java code is to use `System.out.println()` to print values and check the flow of the program.

In this example, the first line `"Before division"` will print, but the second line is never reached because the program crashes due to division by zero:

Example

```
int x = 10;

int y = 0;

System.out.println("Before division");  // Debug output

int result = x / y;

System.out.println("Result: " + result);
```

Result:

```
Before division
Exception in thread "main" java.lang.ArithmeticException: / by zero
```

Debugging with IDEs

Modern IDEs like **IntelliJ IDEA**, **Eclipse**, and **NetBeans** come with built-in debugging tools.

- Set **breakpoints** to pause the program at specific lines
- Step through code line by line
- Inspect variable values in real time

Java Exceptions - Try...Catch

Java Exceptions

As mentioned in the Errors chapter, different types of errors can occur while running a program - such as coding mistakes, invalid input, or unexpected situations.

When an error occurs, Java will normally stop and generate an error message. The technical term for this is: Java will throw an **exception** (throw an error).

Exception Handling (try and catch)

Exception handling lets you catch and handle errors during runtime - so your program doesn't crash.

It uses different keywords:

The `try` statement allows you to define a block of code to be tested for errors while it is being executed.

The `catch` statement allows you to define a block of code to be executed, if an error occurs in the try block.

The `try` and `catch` keywords come in pairs:

Syntax

```
try {
  //  Block of code to try
}
catch(Exception e) {
  //  Block of code to handle errors
}
```

IN WHATSAPP LANGUAGE----

Exception Handling

Exception Handling ka matlab hota hai **program ke error ko handle karna**, taaki jab program run ho aur beech mein koi problem aa jaye toh program **crash na ho**. Java mein kai baar runtime pe errors aa jate hain jaise divide by zero, array ka wrong index use karna, ya null value access karna. Agar hum exception handling use nahi karte, toh program wahi pe ruk jata hai. Exception handling ke through hum `try`, `catch`, `finally`, `throw` aur `throws` ka use karte hain. `try` block mein hum risky code likhte hain, agar error aata hai toh control `catch` block mein chala jata hai jahan hum error ko handle karke proper message show kar dete hain. `finally` block error aaye ya na aaye, hamesha execute hota hai aur mostly resource close karne ke liye use hota hai. `throw` keyword se hum khud se exception create kar sakte hain aur `throws` se exception ko calling method tak forward kar dete hain. Overall,

exception handling program ko **safe, smooth aur user-friendly** banati hai aur normal flow ko maintain karti hai.

AN EXAMPLE OF EXCEPTION HANDLING

This will generate an error, because **myNumbers[10]** does not exist.

```
public class Main {

    int[] myNumbers = {1, 2, 3};

    System.out.println(myNumbers[10]); // error!

  }

}
```

The output will be something like this:

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 10
        at Main.main(Main.java:4)
```

Note: `ArrayIndexOutOfBoundsException` occurs when you try to access an index number that does not exist.

If an error occurs, we can use `try...catch` to catch the error and execute some code to handle it:

Example

```
public class Main {

  public static void main(String[ ] args) {

    try {

      int[] myNumbers = {1, 2, 3};

      System.out.println(myNumbers[10]);

    } catch (Exception e) {

      System.out.println("Something went wrong.");

    }

  } }
```

The output will be:

```
Something went wrong.
```

Finally

The `finally` statement lets you execute code, after `try...catch`, regardless of the result:

Example

```
public class Main {
  public static void main(String[] args) {
    try {
      int[] myNumbers = {1, 2, 3};
      System.out.println(myNumbers[10]);
    } catch (Exception e) {
      System.out.println("Something went wrong.");
    } finally {
      System.out.println("The 'try catch' is finished.");
    }
  }
}
```

The output will be:

```
Something went wrong.
The 'try catch' is finished.
```

IN SIMPLE LANGUAGE----

WHAT IS TRY – CATCH – FINALLY

try – catch – finally Java mein exception handling ke main blocks hote hain, jo program ko **runtime errors se bachate** hain aur program ko **crash hone se rok dete** hain.

🔹 try Block

`try` block mein hum **risky code** likhte hain, jisme error aane ka chance hota hai. Java pehle `try` block ko execute karti hai. Agar yahan koi exception aati hai, toh normal flow ruk jata hai aur control seedha `catch` block mein chala jata hai.

👉 Example: divide by zero, array index error

🔹 catch Block

`catch` block ka kaam hota hai **exception ko handle karna**. Jaise hi `try` block mein error aata hai, matching `catch` block execute hota hai aur program crash hone ke bajaye **proper message show karta hai**.

👉 Ek `try` ke saath **multiple catch** bhi ho sakte hain.

🔹 finally Block

`finally` block **error aaye ya na aaye**, **hamesha execute hota hai**. Iska use mostly **resource close karne** ke liye hota hai jaise file close karna, database connection close karna, etc.

👉 `finally` block optional hota hai, par agar likha hai toh chalega hi.

🔹 Simple Code Example

```java
try {
    int a = 10 / 0;
}
catch (ArithmeticException e) {
    System.out.println("Error: Cannot divide by zero");
}
finally {
    System.out.println("Program ended safely");
}
```

✅ One-Line Summary (WhatsApp)

- **try** → risky code
- **catch** → error handle
- **finally** → always execute